

6. IMAGE ENHANCEMENT USING FREQUENCY DOMAIN FILTERING

AIM:

To study and perform image enhancement using frequency domain filtering.

INTRODUCTION:

The most fundamental relationship between spatial and frequency domains is established by a well-known result called the convolution theorem. The frequency domain may be viewed as a “laboratory” in which we take advantage of the correspondence between the frequency content and image appearance. Some enhancement tasks that would be exponentially difficult or impossible to formulate directly in the spatial domain becomes most trivial in the frequency domain. The fourier transform pairs are given by

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-i2\pi(\frac{ux}{M} + \frac{vy}{N})}$$
$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{i2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Lowpass filters:

It will pass the low frequencies components and will attenuate high frequencies which leads to more smooth images.

Highpass filters:

It will pass high frequencies and will attenuate low frequencies present in the signal and will emphasize details.

IMAGE SMOOTHING USING FREQUENCY DOMAIN FILTER:

Edges and sharp transitions (e.g. noise) in an image contribute to high-frequencies content of its Fourier transform. Hence smoothing is achieved in the frequency domain by high frequency attenuation that is by low pass filtering.

Types of low pass filter:

- ☐ Ideal lowpass filters (sharp)
- ☐ Butterworth lowpass filter
- ☐ Gaussian lowpass filter (smooth)

Basic Steps for Filtering in the Frequency Domain:

Step 1: Multiply the input image by $(-1)^{x+y}$ to center the transform.

Step 2: Compute $F(u,v)$, the DFT of the image from (1).

Step 3: Multiply $F(u,v)$ by a filter function $H(u,v)$.

Step 4: Compute the inverse DFT of the result in (3).

Step 5: Obtain the real part of the result in (4).

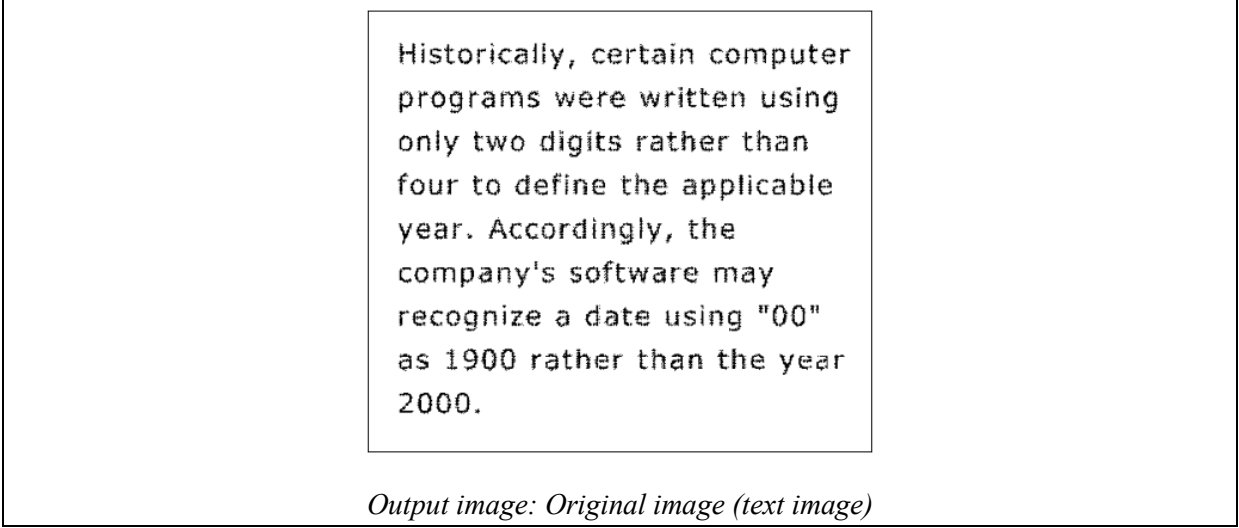
Step 6: Multiply the result in (5) by $(-1)^{x+y}$

Code: to read image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

I= cv2.imread(r"E:\gkbiomed\Image_Processing\Support_materials\IMAGES\DIP3E
IMAGE\DIP3E_Original_Images_CH04\Fig0419(a)(text_gaps_of_1_and_2_pixels).tif",
cv2.IMREAD_GRAYSCALE)

plt.imshow(I, cmap='gray')
plt.axis('off')
```



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

Output image: Original image (text image)

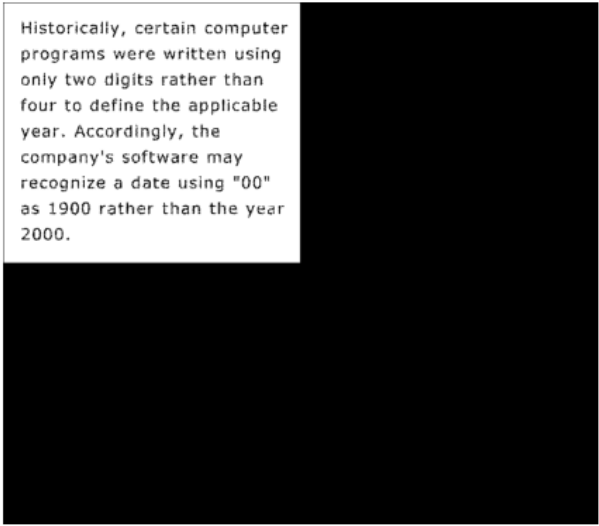
Code: Zero padding the Original image

```
# Original size
M, N = I.shape
P, Q = 2*M, 2*N # padded size
```

```
# Zero-padding

I_padded = np.zeros((P, Q),dtype=np.float32)

I_padded[0:M, 0:N] = I
plt.imshow(I_padded, cmap='gray')
plt.axis('off')
```



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

Output :image of the Zero padded

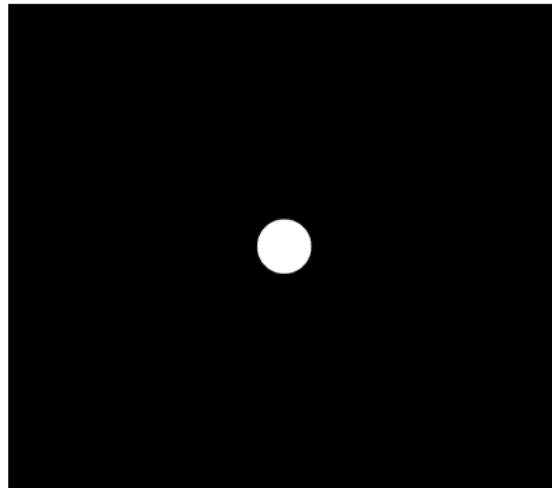
Code: To Create Filter Mask

```
# get cut-off frequency
d0 = float(input("Enter cutoff frequency (d0): "))
# Create Filter mask (ideal)
H = np.zeros((P, Q),dtype=np.float32)
for i in range(P):
    for j in range(Q):
        d = np.sqrt((i - P/2)**2 + (j - Q/2)**2)
        if d <= d0:
            H[i, j] = 1
```

```

        else:
            H[i, j] = 0
plt.imshow(H, cmap='gray')
plt.axis('off')

```



Output : image of the ideal mask

Code: Create Filter Mask(butter worth)

```

# Create Filter mask (Butterworth)
# get cut-off frequency
d0 = float(input("Enter cutoff frequency (d0): "))
# order of the filter
n = float(input("Enter the order (d0): "))
H = np.zeros((P, Q), dtype=np.float32)
for i in range(P):
    for j in range(Q):
        d = np.sqrt((i - P/2)**2 + (j - Q/2)**2)
        H[i, j] = 1/(1+(d/d0)**(2*n))
plt.imshow(H, cmap='gray')
plt.axis('off')

```

Input:

```
Enter cutoff frequency (d0): 30  
Enter the order (d0): 2
```

Output image: Create Filter Mask(butter worth)



Code: Applying FT and Shift to center

```
# Apply FT and shift to center  
F = np.fft.fft2(I_padded)  
F_shifted = np.fft.fftshift(F)  
  
# Apply mask --> Filter  
G = F_shifted * H  
  
# shift back and Apply IFFT  
G_inv = np.fft.ifftshift(G)  
I_filtered_full = np.fft.ifft2(G_inv)  
I_filtered_full = np.real(I_filtered_full)  
plt.imshow(I_filtered_full, cmap='gray')  
plt.axis('off')
```



Output : Applying FT and Shift to center

Code: Crop to original size

```
# Crop back to original size
I_filtered = I_filtered_full[0:M, 0:N]
plt.imshow(I_filtered, cmap='gray')
plt.axis('off')
```



Output : Crop to original size

IDEAL LOW PASS FILTERS:

$D(u,v)$ is the distance from the origin D_0 is the cutoff frequency. The name ideal low pass filter indicates that all frequencies on or inside a circle of radius D_0 are passed without attenuation, whereas all frequencies outside the circle are completely attenuated and it is symmetric about origin. But the sharp cutoff frequencies of an ILPF cannot be realized with electronics component. The major disadvantage of ideal low pass filter is its ringing effect.

$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \leq D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$$

Code: Ideal Low Pass Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH02\Fig0241(c)(einstein high
contrast).tif",cv2.IMREAD_GRAYSCALE)

P, Q = img.shape
d0 = 50

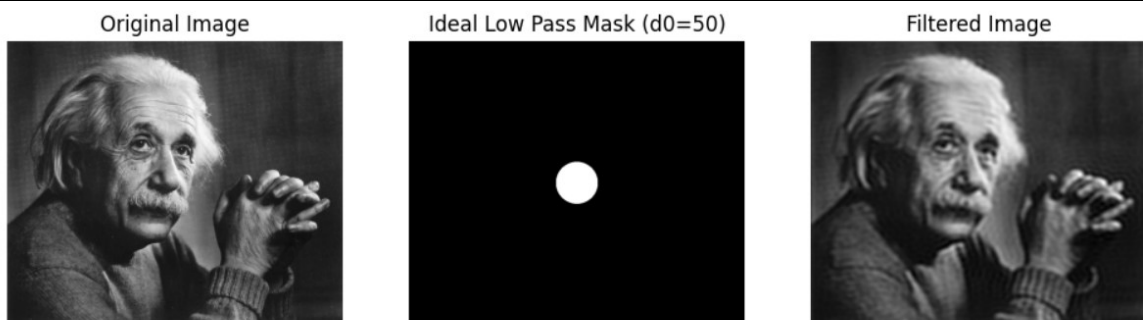
# Create Ideal Low Pass Filter (ILPF)
H = np.zeros((P, Q), dtype=np.float32)
for i in range(P):
    for j in range(Q):
        d = np.sqrt((i - P/2)**2 + (j - Q/2)**2)
        if d <= d0:
            H[i, j] = 1
        else:
            H[i, j] = 0

# FFT and shift
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
```

```

# Apply filter
fshift_filtered = fshift * H
# Inverse FFT
f_ishift = np.fft.ifftshift(fshift_filtered)
img_filtered = np.fft.ifft2(f_ishift)
img_filtered = np.abs(img_filtered)
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(H, cmap='gray')
plt.title(f'Ideal Low Pass Mask (d0={d0})')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(img_filtered, cmap='gray')
plt.title("Filtered Image")
plt.axis('off')
plt.show()

```



Output: The Ideal Low Pass Filter smooths the image by removing high-frequency details, resulting in a blurred appearance.

BUTTERWORTH LOW PASS FILTER:

$D(u,v)$ is the distance from the origin D_0 is the cutoff frequency. n is the order of the filter.

Butterworth LPF does not have a sharp discontinuity that gives a clear cutoff between passed and filtered frequencies. Here ringing effect is significant only for higher order.

$$H(u, v) = \frac{1}{1 + [D(u, v) / D_0]^{2n}}$$

Code: Butterworth Low Pass Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH04\Fig0450(a)(woman_original).ti
f", cv2.IMREAD_GRAYSCALE)

P, Q = img.shape
d0 = 50
n = 2

# Create Butterworth Low Pass Filter (BLPF)
H = np.zeros((P, Q), dtype=np.float32)
for i in range(P):
    for j in range(Q):
        d = np.sqrt((i - P/2)**2 + (j - Q/2)**2)
        H[i, j] = 1 / (1 + (d / d0)**(2 * n))

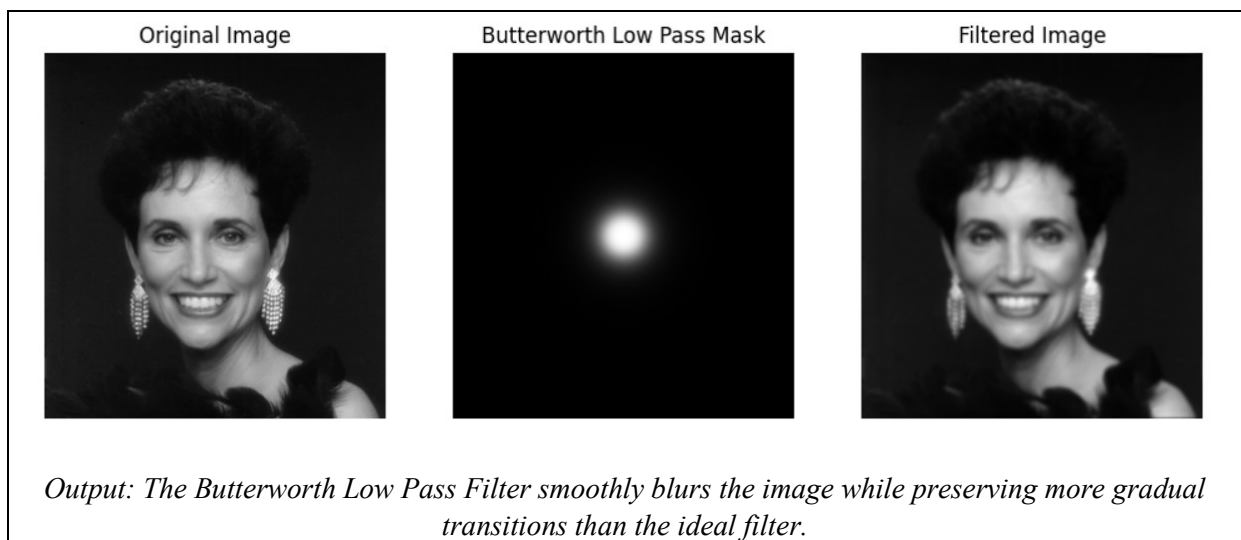
# FFT and shift
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

# Apply filter in frequency domain
fshift_filtered = fshift * H

# Inverse FFT
f_ishift = np.fft.ifftshift(fshift_filtered)
img_filtered = np.fft.ifft2(f_ishift)
img_filtered = np.abs(img_filtered) # magnitude
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
```

```
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(H, cmap='gray')
plt.title("Butterworth Low Pass Mask")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(img_filtered, cmap='gray')
plt.title("Filtered Image")
plt.axis('off')
plt.show()
```



GAUSSIAN LOW PASS FILTER:

$D(u,v)$ is the distance from the origin D_0 is the cutoff frequency. The inverse Fourier transform of the Gaussian Lowpass filter is also Gaussian in the spatial domain. Therefore there is no ringing effect of the GLPF.

$$H(u, v) = e^{-D^2(u,v)/2D_0^2}$$

Code: Gaussian Low Pass Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH04\Fig0417(a)(barbara).tif", cv2.
IMREAD_GRAYSCALE)

P, Q = img.shape
d0 = 50

# Create Gaussian Low Pass Filter (GLPF)
H = np.zeros((P, Q), dtype=np.float32)
for i in range(P):
    for j in range(Q):
        d = np.sqrt((i - P/2)**2 + (j - Q/2)**2)
        H[i, j] = np.exp(-(d**2) / (2 * (d0**2)))

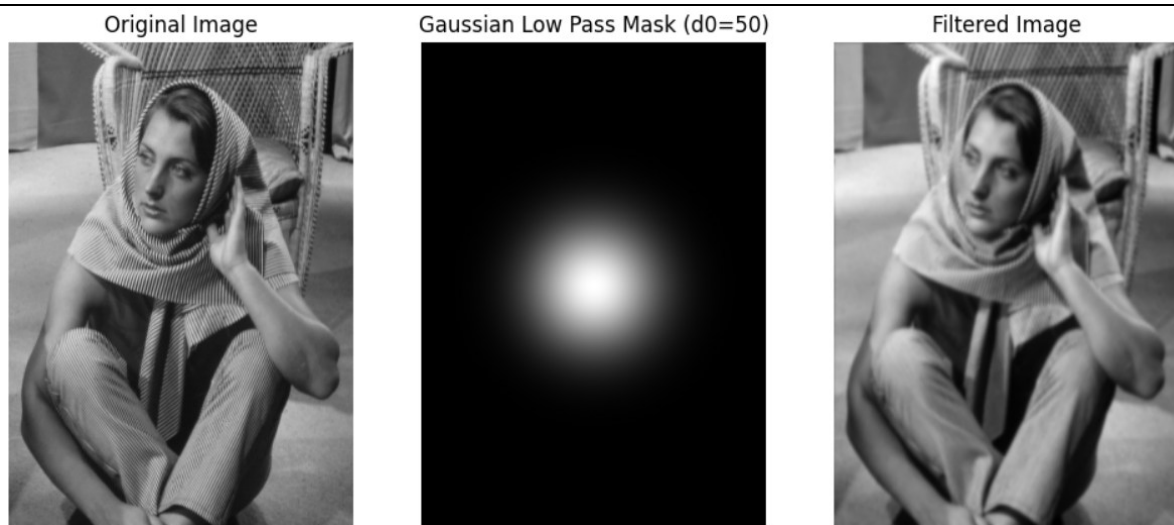
# FFT and shift
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

# Apply filter
fshift_filtered = fshift * H

# Inverse FFT
f_ishift = np.fft.ifftshift(fshift_filtered)
img_filtered = np.fft.ifft2(f_ishift)
img_filtered = np.abs(img_filtered)

plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(H, cmap='gray')
```

```
plt.title(f'Gaussian Low Pass Mask (d0={d0})')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(img_filtered, cmap='gray')
plt.title("Filtered Image")
plt.axis('off')
plt.show()
```



Output: The Gaussian Low Pass Filter smoothly blurs the image with minimal ringing and preserves natural transitions.

CONCLUSION:

- ☐ **Ideal Low Pass Filter:** Strong blurring with a sharp frequency cutoff, but may cause ringing artifacts.
- ☐ **Butterworth Low Pass Filter:** Smooth transition between pass and stop bands, giving moderate blurring with fewer artifacts.
- ☐ **Gaussian Low Pass Filter:** Soft, natural blurring with minimal artifacts, ideal for preserving smooth transitions.